Directive System for Maps' Guided System Based on API

Criterion C-Development

Techniques Used:

1. Use of Express open-source web framework, based on JavaScript, operating under the environment of Node.js alongside with features of Routing, Error Handling, Request and Response Object, Body phrasing.

2. Implementation of MySQL database to manage the data fetch from the frontend direct the way to backend to store.

3. Implementation of Html file to create the GUI using Vision Studio Code[1] coding-software and use its extensive package to construct GUI. I have used the Html tags such as <script>, <tbody>

4. Use of stylesheet consist of CSS file through bootstrap reference link to manage the visual presentation of Html webpage.

5. Use of JSON[2] packages to install all the dependencies of application, modules, and ensure data interchange format

6. Implementation of AMap API to apply the map() related function such as plugin(), lnglat2container(), placesearch.search(), addOverlay() and local.searchNearby()

7. Use the Nodemon[3] npm module completely activate the whole project.

8. Use the XAMPP[4] software to make connection between MYSQL database and server.

[1] Visual Studio Code is a programming software.
[2] Json performed in plain-text format, which allows data to be transmitted among different programming languages.
[3] Nodemon is a command-line tool used to speed up development of Node.js. Monitoring the directory of the project and automatically restart the application when any changes is detected.
[4] XAMPP is an open-source web server packages for running a website on PC, allow coder to test their code locally.

Important Code

Login

The following code creates a connection between the website and the MYSQL database to instore the username and password. By using the code of MYSQL database, execute the inputting information by users from html webpage. Then, check the related username and password match the data in the database or not.

```
const db = mysql.createConnection({
    host: process.env.MYSQL_HOST,
    user: process.env.MYSQL_USER,
    password: process.env.MYSQL_PASSWORD,
    database: process.env.MYSQL_DATABASE
  })
Complexity is 10 It's time to do something...
exports.login = (req, res) => {

    console.log(req.body)
    let account = req.body.account
    let password = req.body.password
    Complexity is 9 It's time to do something...
    db.query(`SELECT * FROM ap_list`,[account,password], function(error, result) {
        if(error){
            console.log(error)
        }

        if(account || password){
            for(i = 0; i < result.length; i++){
                if(account === result[i].account && password === result[i].password){
                    return res.render('home.ejs', {
                    })
                }
            }
            return res.render('login.hbs', {
                message: 'Incorrect password or account'
            })
        }
    })
}
```

Call the code of express framework through find the relative module in node modules file

```
const express = require("express");
const app = express()
const mysql = require("mysql")  271.9k (gzipped: 88.7k)
const dotenv = require("dotenv")  6.4k (gzipped: 2.8k)
const path = require("path")
dotenv.config({path: "./.env"})
```

Server.js:

Determine whether MYSQL database has been successfully connected without any port occupation.

```
20  db.connect((error) =>{
21      if(error){
22          console.log(error)
23      }else{
24          console.log("Database connected...")
25      }
26  })
```

Ensure the port doesn't occupy by other application.

```
43  const port = 3000;
44  app.listen(port, () => {
45      console.log(`Listening at http://localhost:${port}`)
46  })
```

login.hbs:

This file uses the route controller direct the button action to execute data in input and post to the function in *auth.js* document. Once either the username or password mismatch, the login function will export the *message* and alert. User can also use register link direct to register page and enter their information.

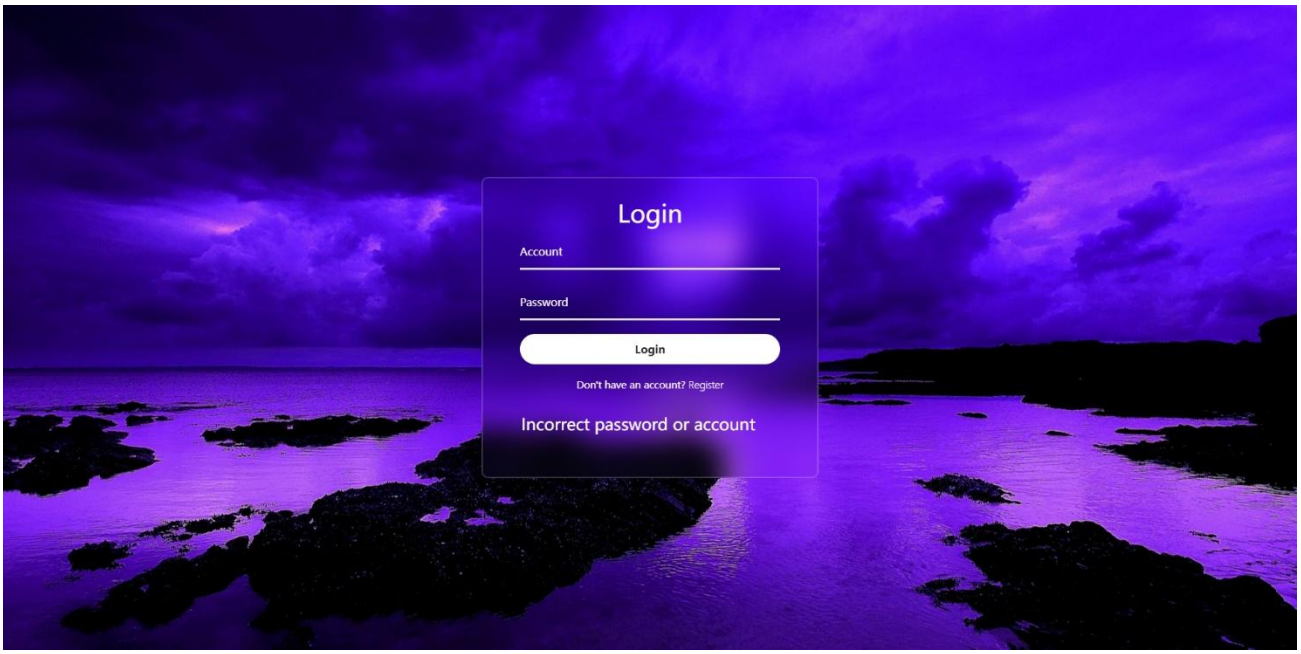
```
142  <div class="wrapper">
143      <div class="dontainer">
144          <h1>Login</h1>
145          <form action="/auth/login" method="POST">
146              <div class="field-holder">
147                  <input type="text" id ="account" name="account" required>
148                  <label for="account">Account</label>
149              </div>
150
151              <div class="field-holder">
152                  <input type="password" id="password" name="password" required>
153                  <label for="password">Password</label>
154              </div>
155
156              <form>
157                  <button type= "submit" class="btn">Login</button>
158              </form>
159              <div class="register-link">
160                  <p>Don't have an account? <a href="/register">Register</a></p>
161              </div>
162          </form>
163      </div>
164      {{#if message}}
165          <h4 class="alert alert-danger mt-4">{{message}}</h4>
166      {{/if}}
167  </div>
```

Graphical User Interface:

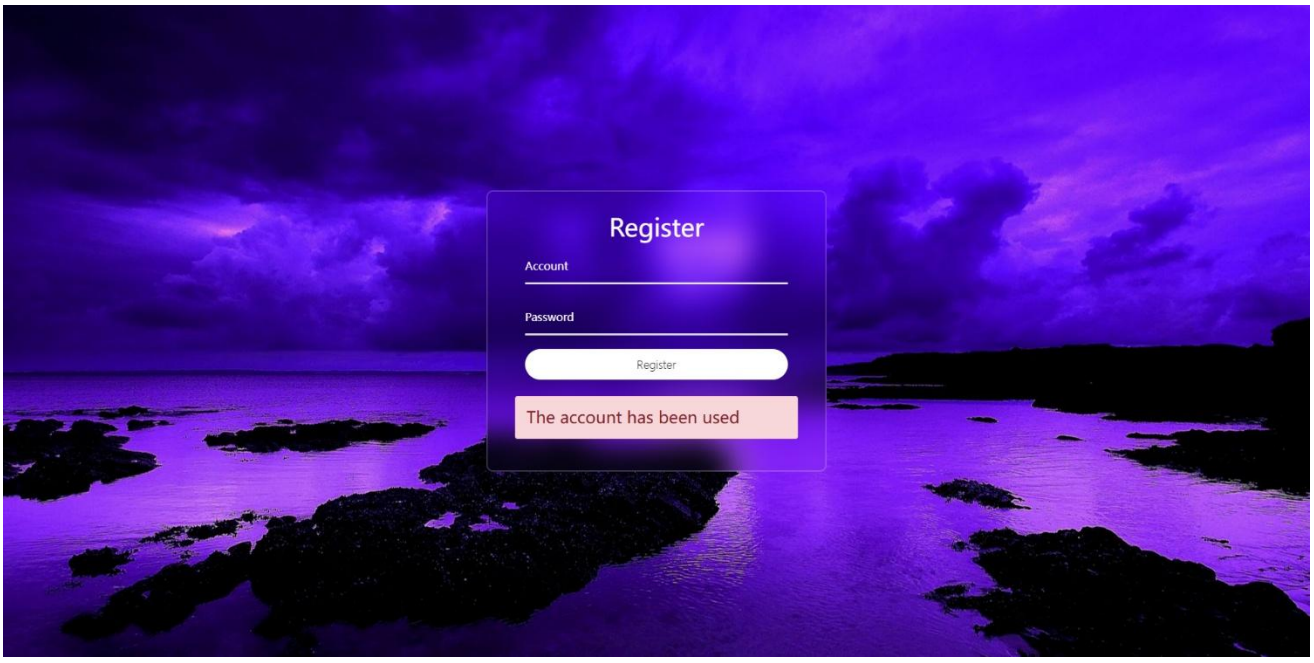
The following are the screenshots of the Html webpage that I have designed. They are linked togther and can be accessed from one another by send the request through http to route, then forward request to appropriate controller to view the templates, handled by <button> and href attribute. Relevant data has been shown where it is affecting some components in the frame.

Login:

The login screen allows user to access two webpages depending on login into the home page or register account for new user.



Register:



.env

Information of used database and host.

```
.env
MYSQL_HOST='127.0.0.1'
MYSQL_USER='root'
MYSQL_PASSWORD=''
MYSQL_DATABASE='ap'
```

auth.js (under controllers file)

By using the same id, js file can execute the type-in account and password from html file. Through comparing in database, analyze whether the account has been used or not. If the password or account mismatch with information in database, there will be alerted information.

```
exports.register = (req, res) => {
    console.log(req.body)
    const {account, password} = req.body

    Complexity is 7 It's time to do something...
    db.query('SELECT account FROM ap_list WHERE account = ?', [account], async (error, result) => {
        if(error){
            console.log(error)
        }

        if(result.length > 0){
            return res.render('register.hbs', {
                message: 'The account has been used'
            })
        }
        Complexity is 3 Everything is cool!
        db.query('INSERT INTO ap_list SET ?', {account: account, password: password}, (error, result) => {
            if(error){
                console.log(error)
            }else{
                console.log(result)
                return res.render('login.hbs', {
                message: 'User registered'
            })
            }
        })
    })
}

```

By declare a .use() method, this short cut will direct render the webpage without wiring long route, such as button with register function, using */auth/register* action can direct to register function in auth.js under

```
<form action="/auth/register" method="POST">
```

```
app.use("/", require("./routes/pages"))
app.use("/auth", require("./routes/auth"))
app.use("/js", express.static(__dirname + " /public/js"))
app.use("/css", express.static(__dirname + " /public/css"))
```

Export the post method (under routes file):

All the complex function will be written under auth.js under controllers file, through variable authcontroller, the exports function can be directed from controllers file.

```
var express = require("express")
const authcontroller = require("../controllers/auth.js")
var router = express.Router()

router.post('/register', authcontroller.register)
router.post('/login', authcontroller.login)
router.post('/history', authcontroller.history)
router.post('/favourite', authcontroller.favourite)
router.post('/edit_sample_data', authcontroller.edit_sample_data)
router.post('/add_sample_data', authcontroller.add_sample_data)
router.post('/edit_sample_data_cal', authcontroller.edit_sample_data_cal)
router.post('/add_sample_data_cal', authcontroller.add_sample_data_cal)
```

Direct to controller file

Get the webpage:

Simple function will construct in auth.js under routes file. Through action or method 'get' from webpages, this file can direct users to relevant webpages.

```
const express = require("express")
const authcontroller = require("../controllers/auth.js");
const mysql = require("mysql")  271.9k (gzipped: 88.7k)
const router = express.Router()

const db = mysql.createConnection({
    host: process.env.MYSQL_HOST,
    user: process.env.MYSQL_USER,
    password: process.env.MYSQL_PASSWORD,
    database: process.env.MYSQL_DATABASE
  })


router.get("/login", (req, res) => {
    res.render("login.hbs")
})
router.get("/register", (req, res) => {
    res.render("register.hbs")
})
router.get("/map", (req, res) => {
    res.render("map.hbs")
})
router.get("/", (req, res) => {
    res.render("initial.ejs")
})
router.get("/map_route", (req, res) => {
    res.render("map_route.ejs")
})
router.get("/home", (req, res) => {
    res.render("home.ejs")
})
router.get("/StudyMode", (req, res) => {
    res.render("Studymode.ejs")
})
router.get("/CausalMode", (req, res) => {
    res.render("Causalmode.ejs")
})
router.get("/Recommendation", (req, res) => {
    res.render("Recommendation.ejs")
})
```

Through route '/login', webpage login.hbs can be accessed.

File logic:

File name	Contained sub-file
Storage	controllers
	node-modules
	public
	routes
	views
	.env
	server.js
controllers	auth.js
node_modules	Different modules
public	Different images
	style.css
routes	auth.js
	pages.js
views	Bybus.ejs
	Bybus(cal).ejs
	Bycar.ejs
	Bycar(cal).ejs
	Causalmode.ejs
	Favourite.ejs
	home.ejs
	initial.ejs
	login.hbs
	map.hbs
	locatedip.ejs
	Recommendation.ejs
	register.hbs
	sample_data.ejs
	searching.ejs
	studymode.ejs
	styling.css
	surroundings.ejs
	surroundings(Causal).ejs
	Traffic.ejs

MYSQL Database (ap)

Tables' name	Fields' name
ap_list	id
	account
	password
favourite	idd
	tipinput
	time
history	idd
	tipinput
	time

sample_data:

When route of sample_data is gotten from the History button, the function will go through the data in history table and sent to the frontend, then render the sample_data.ejs file.

```
Complexity is 4 Everything is cool!
router.get("/sample_data", function(request, response, next){

    var query = "SELECT * FROM history";

    Complexity is 3 Everything is cool!
    db.query(query, function(error, data){

        if(error)
        {
            throw error;
        }
        else
        {
            response.render('sample_data.ejs', {title:'History Searched', action:'list', sampleData:data, message:request.flash('success')});
        }

    });

});
```

```
<form action="/sample_data">
    <button type="submit" class="atn">History</button>
</form>
```

sample_data.ejs:

The .ejs file will use embedded dynamic content to display the data inside history table. The data will be organized according to their id, searching history and time.

```
<tr>
    <th>id</th>
    <th>Searched destination</th>
    <th>Time</th>
    <th>Action</th>
</tr>
<%
if(sampleData.length > 0)
{
    sampleData.forEach(function(data){
%>
<tr>
    <td><%= data.idd %></td>
    <td><%= data.tipinput %></td>
    <td><%= data.time %></td>
    <td>
        <a href="/routes/pages.js/edit/<%= data.idd %>" class="btn btn-primary btn-sm">Edit</a>
        <a href="/routes/pages.js/delete/<%= data.idd %>" class="btn btn-danger btn-sm">Delete</a>
    </td>
</tr>
<%
    });
}
else
{
%>
<tr>
    <td colspan="5">No Data Found</td>
</tr>
<%
}
%>
```

```
<a href="/routes/pages.js/add" class="btn btn-success btn-sm float-end">Add</a>
<a href="/home" class="btn btn-success btn-sm float-end" style="position: relative; right: 30px;">Back</a>
```

Edit function:

When the delete button is pressed, a specific route will be directed through get method, and review the specific piece of data through the id obtain from .ejs file and match within database.

```
router.get('/routes/pages.js/edit/:idd', function(request, response, next){

    var idd = request.params.idd;

    db.query(`SELECT * FROM history WHERE idd = "${idd}"`, function(error, data){

        response.render('sample_data.ejs', {title: 'Edit MySQL Table Data', action:'edit', sampleData:data[0]});

    });

});
```

In the editing form, users are allowed to edit the destination with relative id shown within the table.

```
<% } else if(action == 'edit') { %>
<div class="card">
    <div class="card-header">Sample Form</div>
    <div class="card-body">
        <form method="POST" action="/auth/edit_sample_data">
            <div class="row">
                <div class="col-md-6">
                    <div class="mb-3">
                        <label>id</label>
```

```
<input type="submit" name="submit_button" class="btn btn-primary" value="Edit"
```

.edit_sample_data:

Request the id and history of the data through name and id attribute from frontend, and use the 'UPDATE' MYSQL statement to edit the related history.

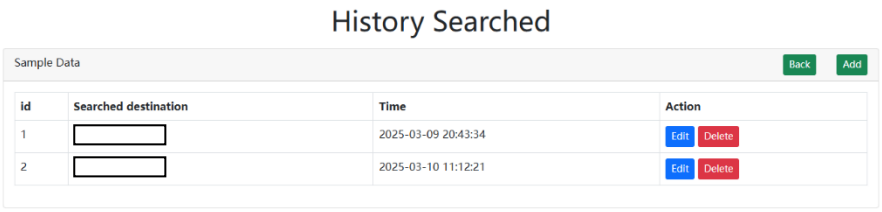
```
exports.edit_sample_data = (request, response) => {
    var idd = request.params.first_name;
    var idd = request.body.first_name;
    var last_name = request.body.last_name;
    var query = `UPDATE history SET tipinput = "${last_name}" WHERE idd = "${idd}"`;

    Complexity is 3 Everything is cool!
    db.query(query, function(error, data){

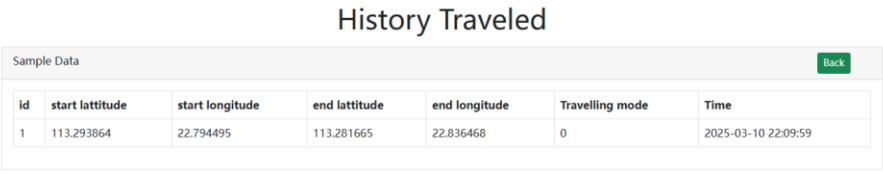
        if(error)
        {
            throw error;
        }
        else
        {
            request.flash('success', 'Sample Data Edited');
            response.redirect('/sample_data');
        }
    });
}
```

History:

Users can check their previous searching history though table connect to MYSQL database.



Travelling History:



Add function:

As users want add history, the pressed button provide a route direct to the ‘add’ webpage (the action ‘add’ is post to the webpage and chose the related form).

```
93  router.get("/routes/pages.js/add", function(request, response, next){
94
95      response.render("sample_data.ejs", {title:'Insert Data into MySQL', action:'add'});
96
97  });
```

.add_sample_data:

The added history will request from frontend .ejs file and store in last_name. Through MYSQL statement INSERT, functions allow users to add history into specific table.

```
117 exports.add_sample_data = (request, response) => {
118     var last_name = request.body.last_name;
119
120     var query = `INSERT INTO history (tipinput) VALUES ("${last_name}")`;
121
        Complexity is 3 Everything is cool!
122     db.query(query, function(error, data){
123
124         if(error)
125         {
126             throw error;
127         }
128         else
129         {
130             request.flash('success', 'Sample Data Inserted');
131             response.redirect("/sample_data");
132         }
133
134     });
135 }
```

The ‘Add’ button in the form have a action to post the add_sample_data function under auth.js file.

```
18              <% if(action == 'add') { %>
19
20              <div class="card">
21                  <div class="card-header">Sample Form</div>
22                  <div class="card-body">
23                      <form method="POST" action="/auth/add_sample_data">
24                          <div class="row">
25                              <div class="col-md-6">
26                                  <div class="mb-3">
27                                      <label>Searched destination</label>
28                                      <input type="text" name="last_name" id="last_name" class="form-control" />
```

```
<input type="submit" name="submit_button" class="btn btn-primary" value="Add" />
```

.forEach():

The webpages can have several pages within one route, linking them through buttons and addEventListener() function to change webpages. When clicking the button to turn pages, data-target is used to store the id of targeted webpage.

```
209         const pageLinks = document.querySelectorAll('.pagination a');
210
211         pageLinks.forEach(link => {
212             link.addEventListener('click', (event) => {
213                 event.preventDefault();
214
215                 const targetPage = link.getAttribute('data-target');
216                 const currentPage = link.closest('section');
217
                    currentPage.style.display = 'none';

                    document.getElementById(targetPage).style.display = 'block';
221             });
222         });
223         document.getElementById('page1').style.display = 'block';
```

Default first webpage.

Hide webpages

Show webpage

By posting the method favourite, the tipinput will request the content in the text area in frontend and submit to favourite table using INSERT INTO statement.

```
137         <form action="/auth/favourite" method="post">
138             <input id="tipinput" name="tipinput" class="tipinput"
139             <button type="submit" class="button">Store</button>
```

When history is successfully added into the table, the webpage will refresh.

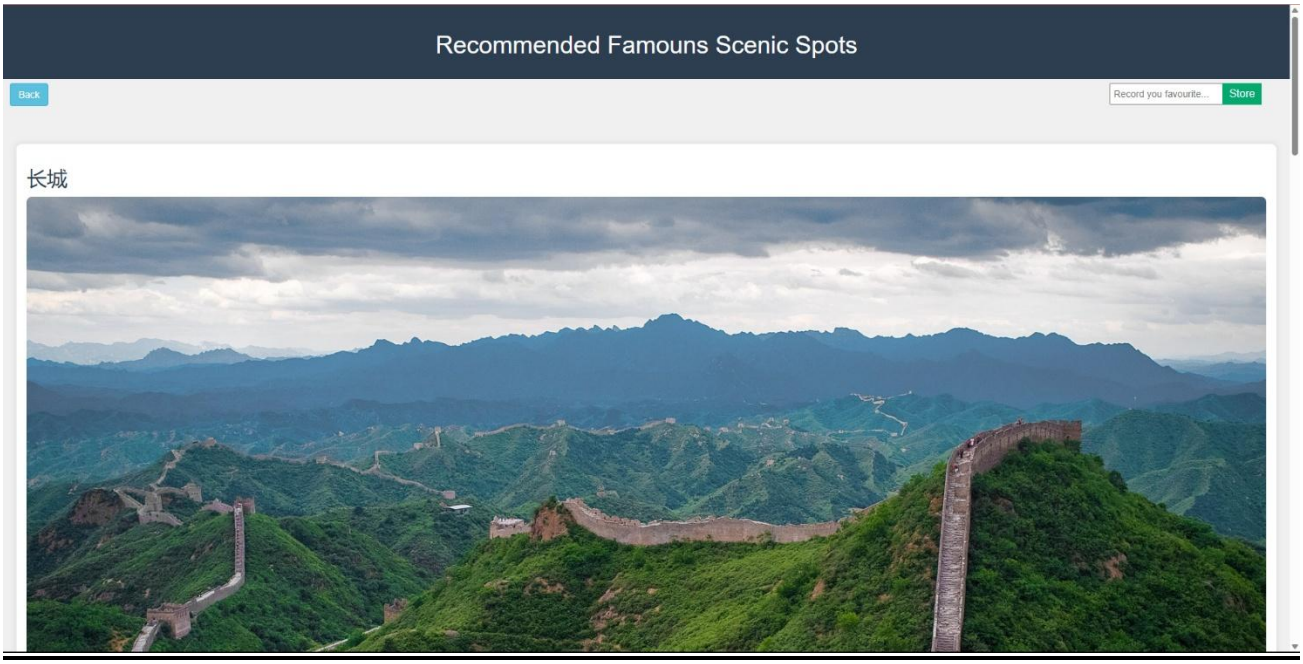
```
83  exports.favourite = (req, res) => {
84      console.log(req.body)
85      const {tipinput} = req.body
86
        Complexity is 3 Everything is cool!
87      db.query('INSERT INTO favourite SET ?', {tipinput: tipinput}, (error, result) => {
88          if(error){
89              console.log(error)
90          }else{
91              console.log(result)
92              return res.render('Recommendation.ejs')
93          }
94      })
95  }
```

The users can access to more information about the destination by click on the link.

```
<h2>长城</h2>
<img src="https://upload.wikimedia.org/wikipedia/commons/thumb/2/23/The_Great_Wall_of_Ch
<a href="https://zh.wikipedia.org/wiki/%E9%95%BF%E5%9F%8E" class="more-link">更多</a>
```

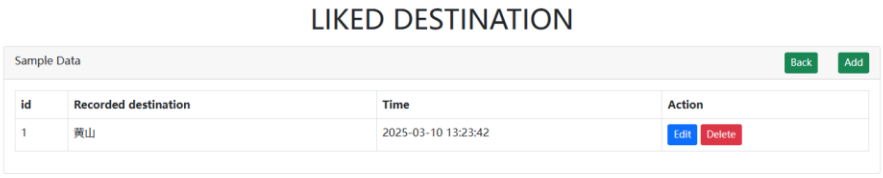
Recommendation:

Users enable to check recommended scenic plot.



Favourite:

Users enable to store their liked destination obtain from the recommendation webpage.



AMap.Map():

Map object class, which encapsulates interfaces such as map property settings, layer changes, and event interactions. By implementing the security code and API-key from the AMap Developer platform, different map function can be accessed.

```
<script type="text/javascript">
window._AMapSecurityConfig = {
  securityJsCode: "f6b242cd04daebb47685da20c645bd53",
};
</script>
<script type="text/javascript" src="https://webapi.amap.com/maps?v=2.0&key=c3f65892c3589481fd26d92b375055e0&plugin=AMap.Driving"></script>
```

.AutoComplete():

According to the input keyword prompt matching information, Poi type and city can be used as input prompt restriction conditions. Users can retrieve and display it through a custom callback function.

.PlaceSearch():

Location search service plug-in, providing location query services for a specific area.

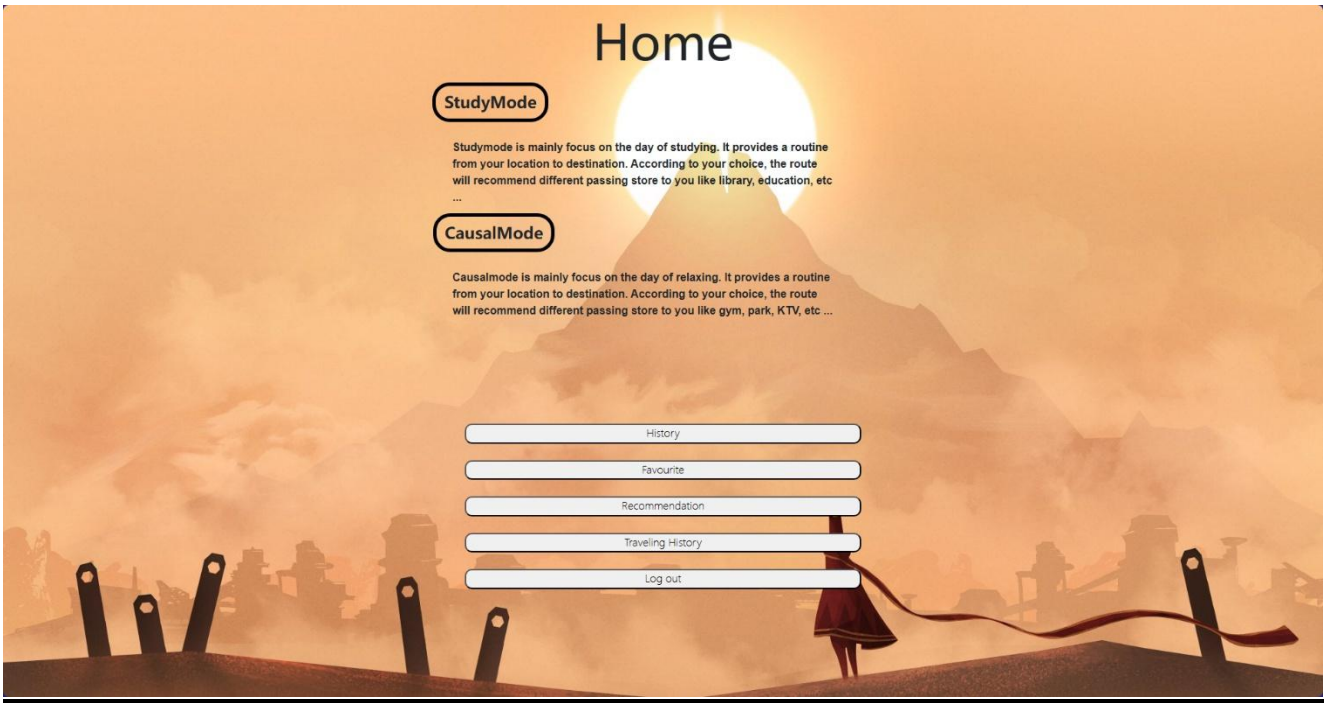
```
<script type="text/javascript">
    //Load the map
    var map = new AMap.Map("container", {
        resizeEnable: true,
        zoom: 13
    });
    //Enter POI
    var autoOptions = {
        input: "tipinput"
    };

    AMap.plugin(['AMap.PlaceSearch','AMap.AutoComplete'], function(){
        var auto = new AMap.AutoComplete(autoOptions);
        var placeSearch = new AMap.PlaceSearch({
            map: map
        });  //Construct the pleace search method class
        auto.on("select", select);//Listen to the event, when a record is selected, it will trigger
        function select(e) {
            placeSearch.setCity(e.poi.adcode);
            placeSearch.search(e.poi.name);  //Keyword search
        }
    });
```

The Forward button will record the search destination of users to database. By directing to auth.js file, to post function history, then using INSERT INTO statement in MYSQL.

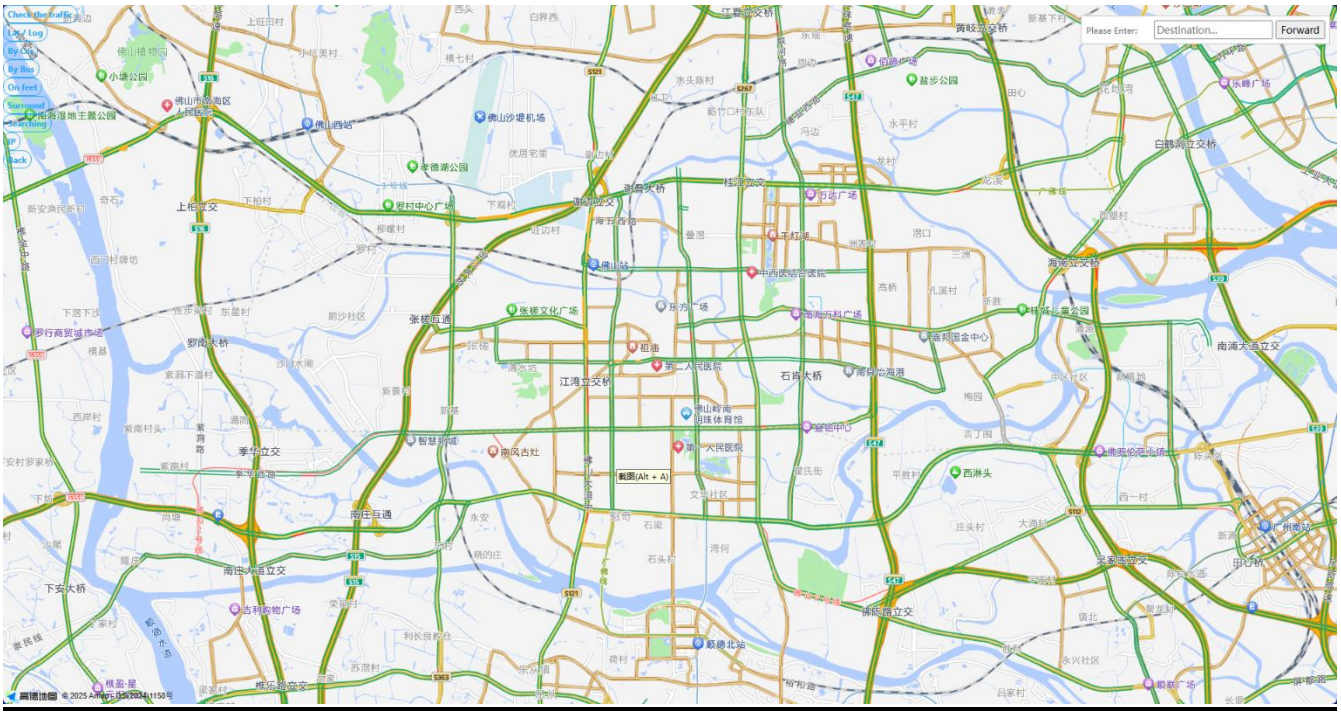
```
<form action="/auth/history" method="POST">
    <label for="tipinput">Please Enter:</label>
    <input id="tipinput" placeholder="Destination..." name="tipinput" class="tipinput">
    <button type="submit">Forward</button>
</form>
```

Home:



Map(main):

This interface allow user to choose the function they like, or directly search the place they want to travel.



AMap.TileLayer.Traffic():

Real-time traffic layer class, inherited from TileLayer.

AMap.TileLayer.RoadNet():

Road network layer, showing road information.

```
var trafficLayer = new AMap.TileLayer.Traffic({
    zIndex: 11
});

var roadNetLayer = new AMap.TileLayer.RoadNet({
    zIndex: 10
});


var layerGroup = new AMap.LayerGroup([trafficLayer, roadNetLayer]);
layerGroup.setMap(map);
```

AMap.Geocoder():

Geocoding and reverse geocoding class, used for conversion between address description and latitude and longitude coordinates. Users can get query results through callback functions.

Geocoder.getLocation():

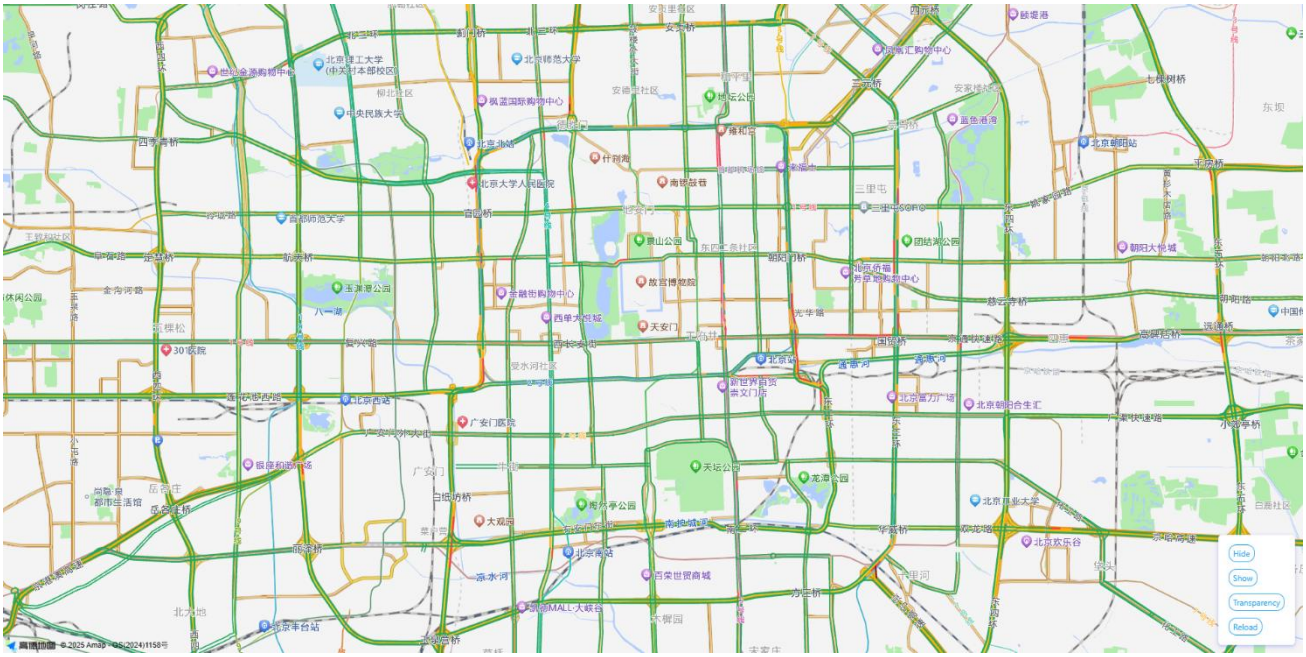
Convert address information into Amap latitude and longitude coordinate information.

```
var geocoder = new AMap.Geocoder({
    city: "",
});

var marker = new AMap.Marker();
function geoCode() {
    var address  = document.getElementById('address').value;
    geocoder.getLocation(address, function(status, result) {
        if (status === 'complete'&&result.geocodes.length) {
            var lnglat = result.geocodes[0].location
            document.getElementById('lnglat').value = lnglat;
            marker.setPosition(lnglat);
            map.add(marker);
            map.setFitView(marker);
        }else{
            log.error('Failed to get the location information');
        }
    });
}
document.getElementById("geo").onclick = geoCode;
document.getElementById('address').onkeydown = function(e) {
    if (e.keyCode === 13) {
        geoCode();
        return false;
    }
    return true;
};
```

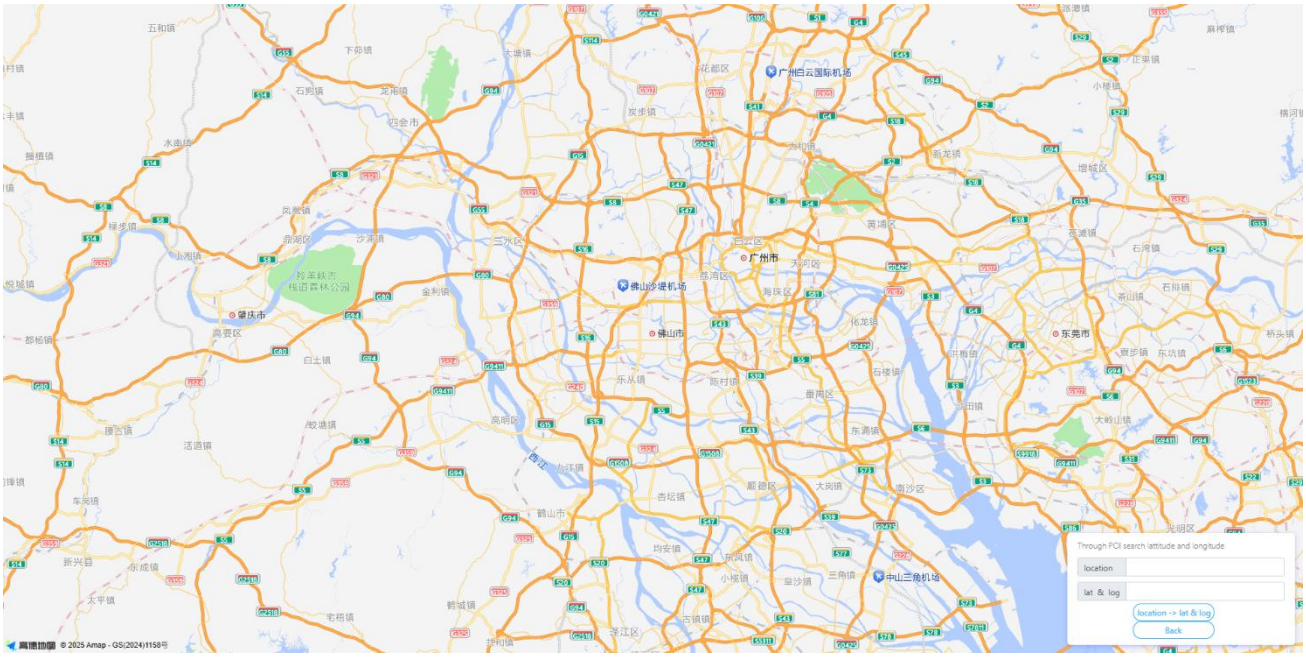
Traffic:

Provide consumer to check the traffic conditions from their location through trace their IP address.



Lat/Log:

Through user's POI (point of interest), the latitude and longitude of relevant destination or starting point will show and used to make-up the navigation routes.



conductSearch():

Forward button has an onclick function. Providing id for each latitude and longitude input of the starting point and destination.

```
<button onclick="conductSearch()">Forward</button>
<label for="">Begin Lat: </label>
<input id="tipinput"  value="">
```

Store the value of input in different variable, and use alert statement for users to ensure their input is correct.

AMap.Driving():

The parameter of the AMap. Driving constructor is the DrivingOptions object. DrivingOptions allows setting driving strategies and return information details.

driving.search():

According to the latitude and longitude or name of the starting point and destination, realizing driving route planning.

Map.setCenter():

Set the center of map after find the route.

```
function conductSearch() {

    var value1 = document.getElementById('tipinput').value;
    var value2 = document.getElementById('tepinput').value;
    var value3 = document.getElementById('topinput').value;
    var value4 = document.getElementById('twpinput').value;
    var value5 = document.getElementById('choice').value;
    alert(value1+ "," + value2 + "," + value3 + "," + value4 + "," + value5);

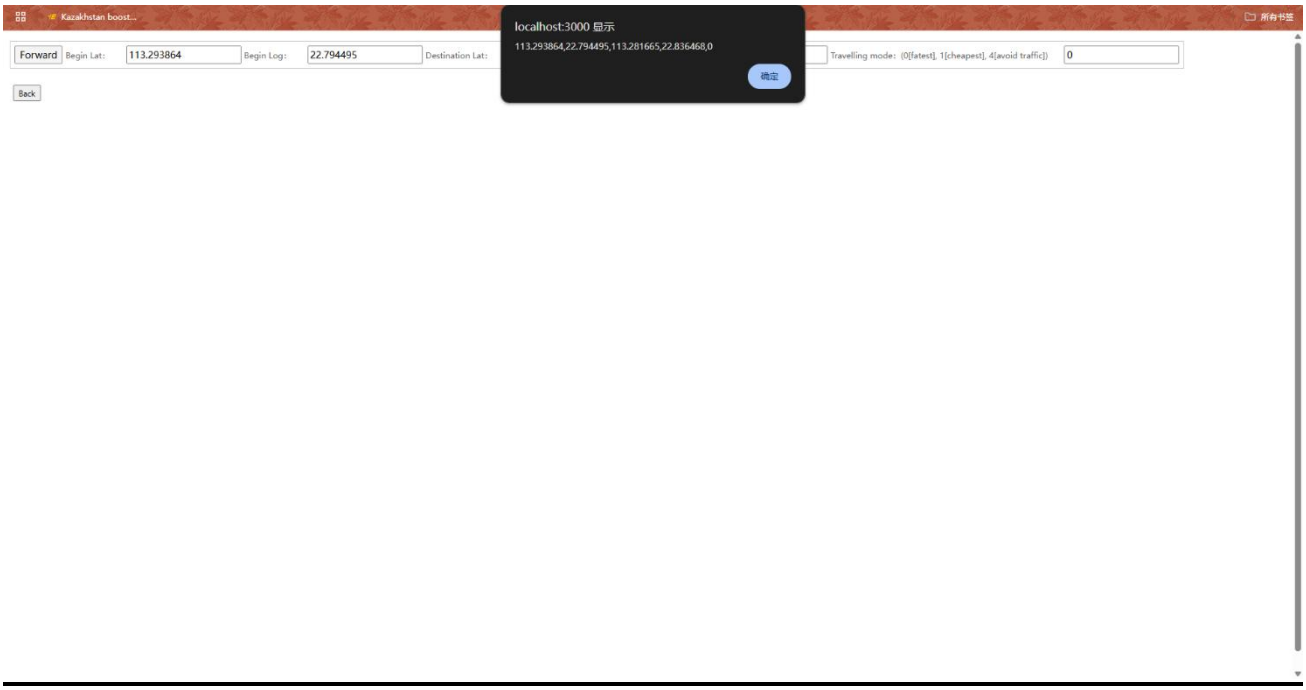
    //Basic loaded map
    var map = new AMap.Map("container", {
      resizeEnable: true,
      center: [116.397428, 39.90923],//Map center
      zoom: 13 //Display zoom level
    });

    //Construct route and navigation class
    var driving = new AMap.Driving({
      map: map,
      panel: "panel",
      policy: value5,
    });

      driving.search(new AMap.LngLat(value1,value2), new AMap.LngLat(value3,value4), function (status, result) {
      // result is the type of return  https://lbs.amap.com/api/javascript-api/reference/route-search#m_DrivingRes
        if (status === 'complete') {
        // log.success(autoOptions)
          log.success('successfully create car route')
          map.setCenter([value1, value2])
        } else {
          log.error('Fail to obtain the car route data: ' + result)
        }
      });
}
```

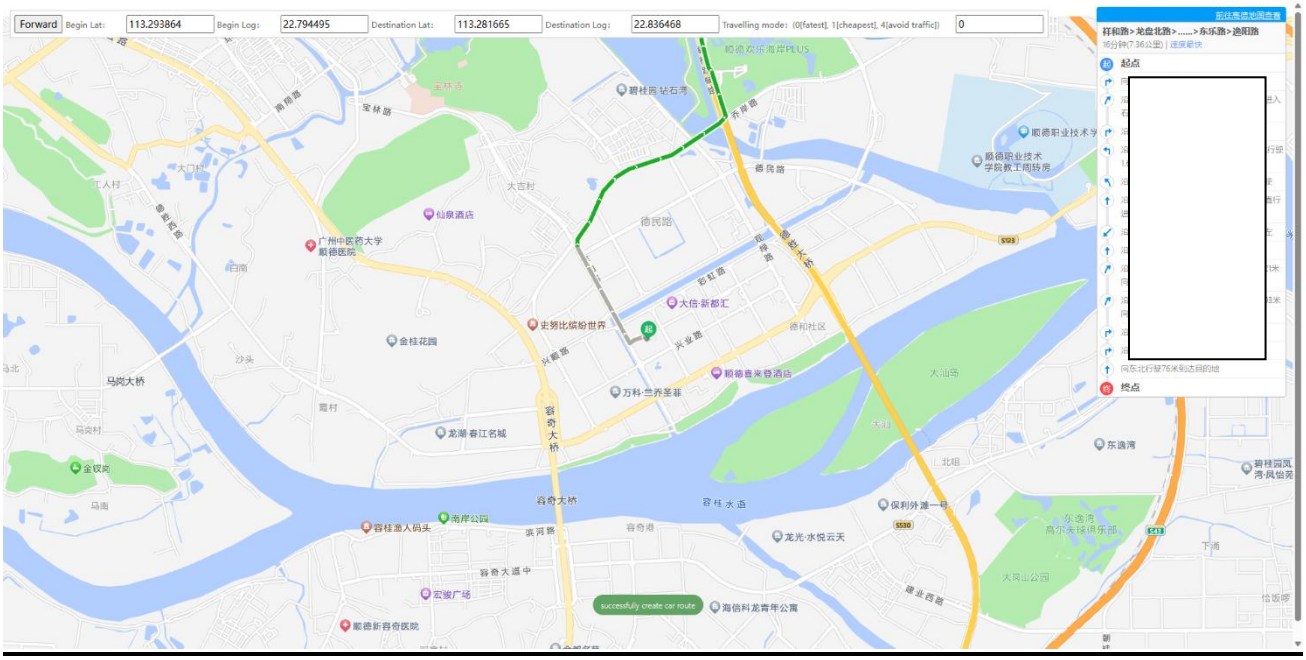
By car:

By inputting the latitude and longitude, users can ensure their starting point and destination, and the routes will be formed.



By car:

The routes will be shown and in the fasted way.



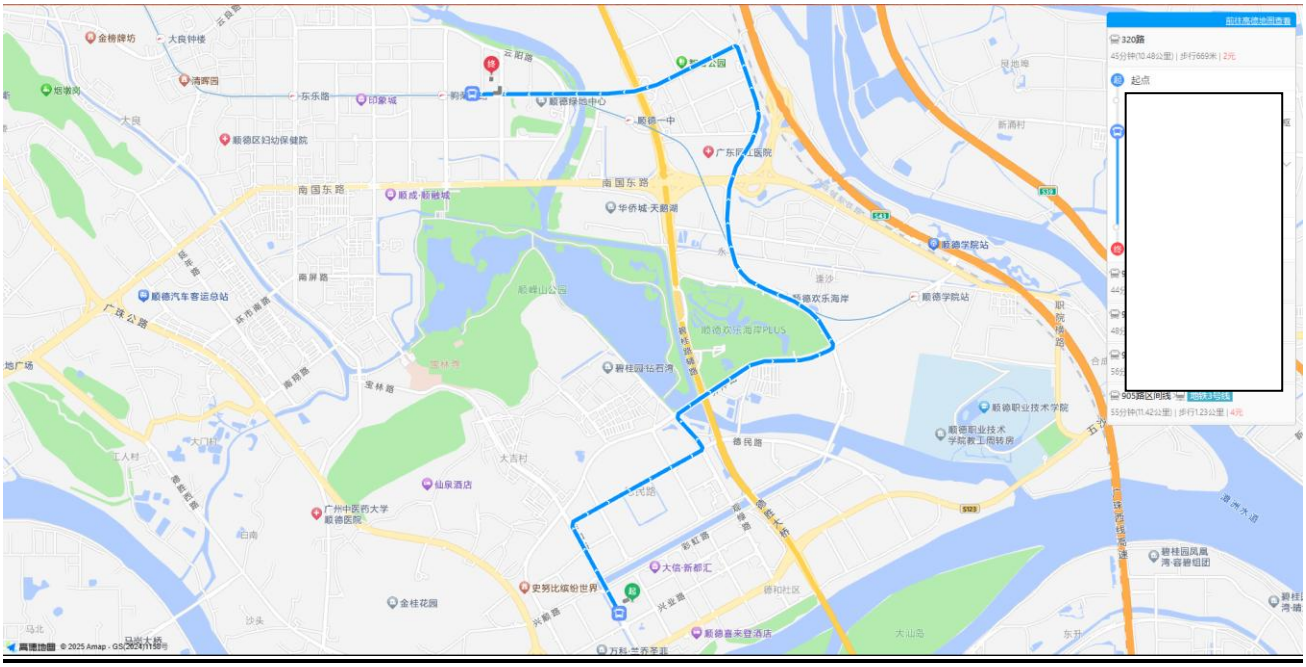
By bus:

Same with above by car.



By bus:

Providing different routes with different amount of bus fee.



AMap.Transfer():

Bus transfer service, providing starting point bus route planning service. The results returned by the bus transfer query integrate walking information. If the transfer type of the transfer segment (Segment) is the subway "SUBWAY", it will contain at least one subway entrance information.

transfer.search():

Perform bus transfer query based on the starting and ending point coordinates.

```
var transOptions = {
  map: map,
  city: '佛山市',
  panel: 'panel',
  //cityd:'',
  policy: AMap.TransferPolicy.value5,
};
//Construct bus transfer class
var transfer = new AMap.Transfer(transOptions);
//Construct route according to the starting point and destination point
transfer.search(new AMap.LngLat(value1, value2), new AMap.LngLat(value3, value4), function (status, result) {
  // result is the type of data  https://lbs.amap.com/api/javascript-api/reference/route-search#m_TransferResu
  if (status === 'complete') {
    map.setCenter([value1, value2])
    log.success('successfully create bus route')
  } else {
    log.error('Fail to create the bus route' + result)
  }
});
```

AMap.Walking():

The walking route planning service provides driving navigation route query function for the starting and ending coordinates. The parameter of the AMap.Driving constructor is the DrivingOptions object.

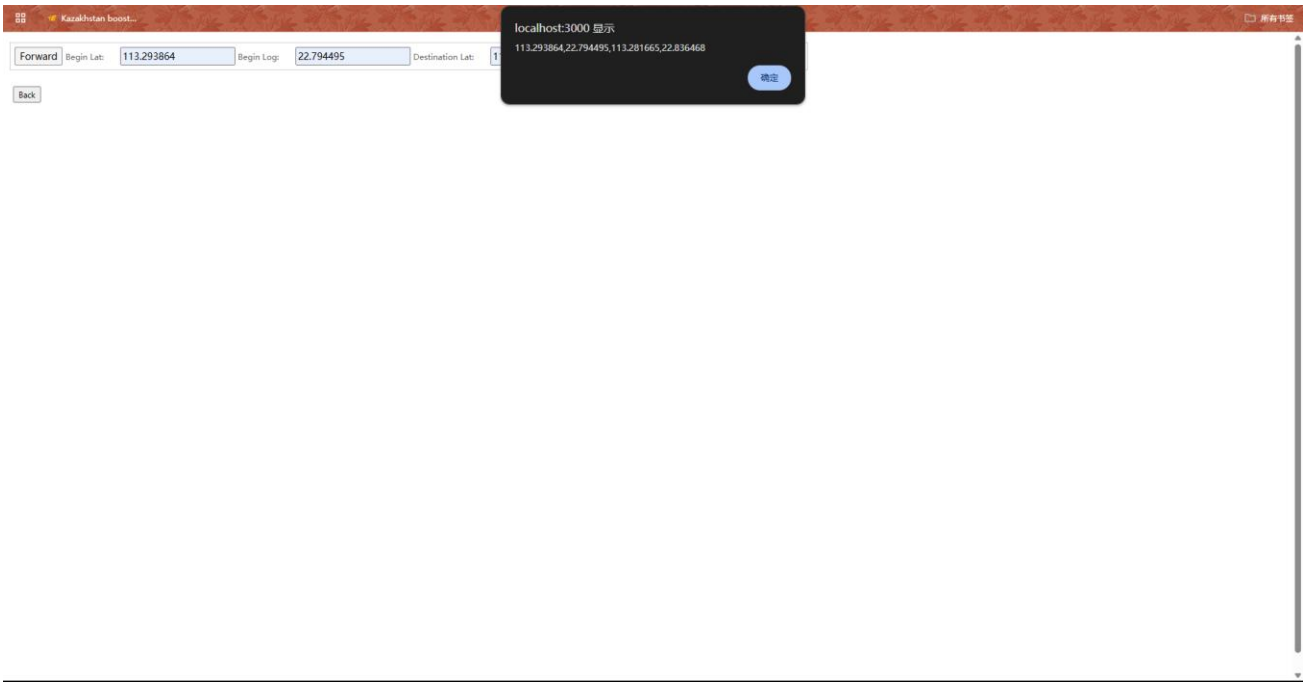
walking.search():

According to the starting and ending coordinates, implement walking route planning.

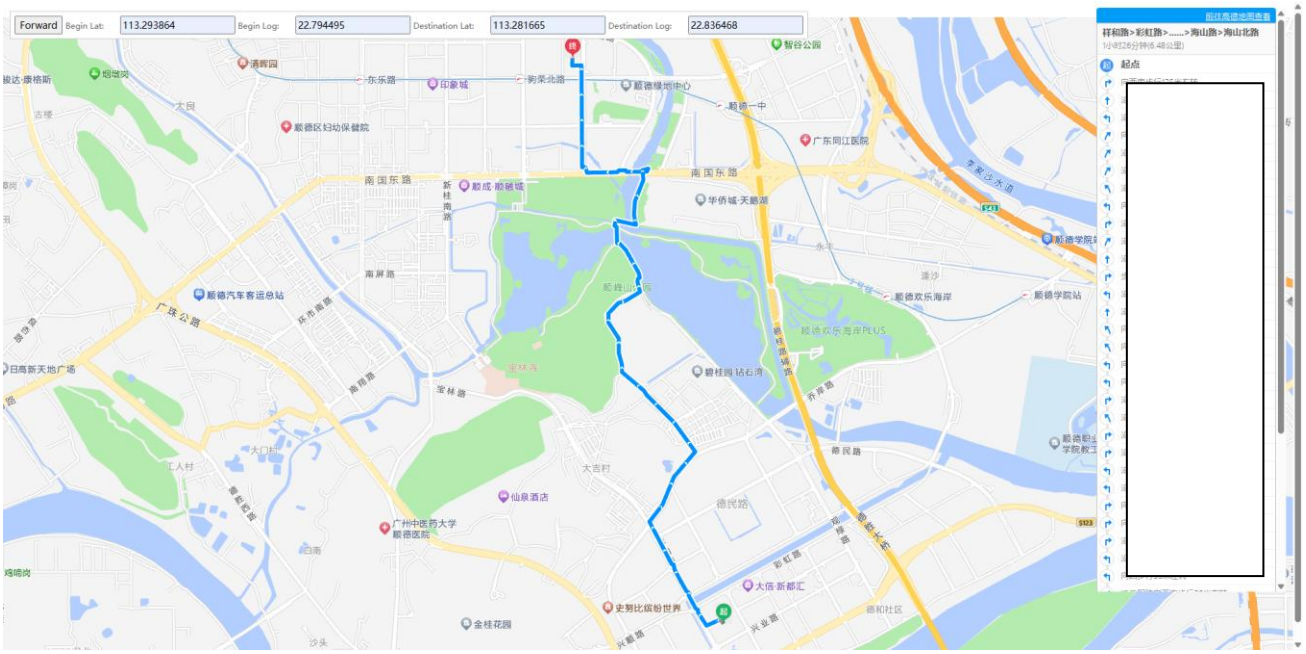
```
var map = new AMap.Map("container", {
    resizeEnable: true,
    center: [value1, value2],//map center
    zoom: 13 //display the zoom level
});

//navigation of walking route
var walking = new AMap.Walking({
    map: map,
    panel: "panel"
});
//Plan walking route according to the given coordinates
walking.search([value1, value2], [value3, value4], function(status, result) {
    // result is the relevant data  https://lbs.amap.com/api/javascript-api/reference/route-search
    if (status === 'complete') {
        log.success('Successfully create walking route')
    } else {
        log.error('Fail to construct the walking route' + result)
    }
});
```

On feet:


Kazakhstan boost...
所有书签
Forward
Begin Lat:
113.293864
Begin Log:
22.794495
Destination Lat:
localhost:3000 显示
113.293864,22.794495,113.281665,22.836468
确定
Back

On feet:


Forward
Begin Lat:
113.293864
Begin Log:
22.794495
Destination Lat:
113.281665
Destination Log:
22.836468
起点

placesearch.searchNearby():

Perform surrounding query based on the center point's latitude and longitude, radius, and keywords. The radius value range is: 0-50000 meters.

```
var cpoint = [value1,value2]; //Center point
placeSearch.searchNearBy('', cpoint, 500, function(status, result) {

});
```

dealSearch():

Through the 'Search' button can access to dealSearch() function. Function will execute the type-in result from webpage and get value through getElementById(), then display to users.

```
<input type="text" id="tipinput" placeholder="Enter your location for weather...">
<input id='stationKeyWord' type="text" value='' placeholder="Bus stop..." >
<button id="tip" onclick="dealSearch()" type="submit">Search</button>
```

geolocation.getCurrentPosition():

It's a callback function is passed to handle the positioning result. Determine whether it's 'complete' or error.

```
function dealSearch(){

    var value6 = document.getElementById('tipinput').value;
    var value7 = document.getElementById('stationKeyWord').value;
    alert("Your position is " + value6 + "\n" + "The bus stop is around " + value7);

    var map = new AMap.Map('container', {
        resizeEnable: true
    });
    AMap.plugin('AMap.Geolocation', function() {
        var geolocation = new AMap.Geolocation({
            enableHighAccuracy: true,//high accuracy located
            timeout: 10000,          //exceeding 10s will be considered as failure
            position:'RB',    //location of the located button
            offset: [10, 20], //location button and the top left corner
            zoomToAccuracy: true,   //zoom to the location located

        });
        map.addControl(geolocation);
        geolocation.getCurrentPosition(function(status,result){
            if(status=='complete'){
                onComplete(result)
            }else{
                onError(result)
            }
        });
    });
```

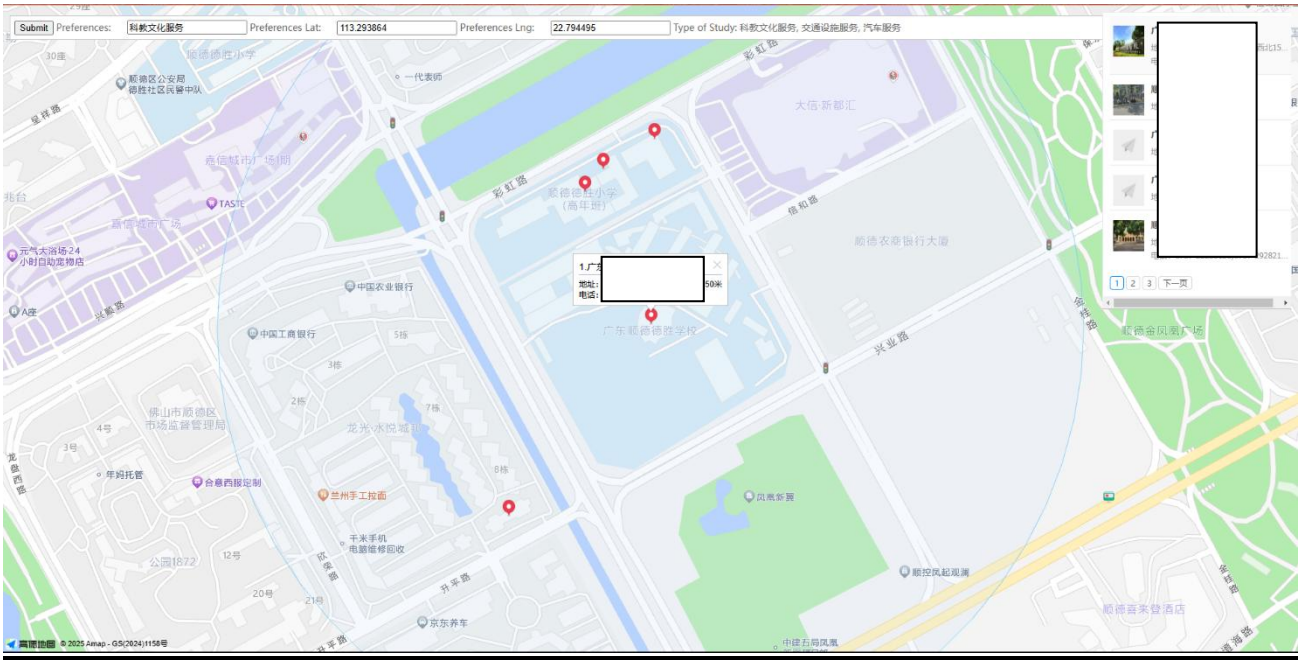
Surroundings:

By inputting the related latitude and longitude, and users' preference, a series relevant location will be shown depend on the modes that users choose (Study or Causal).



Surroundings:

From centre point 500 meters as radius nearby, all relevant location will be shown and pointing on location, brief information will be displayed.



onComplete(): Analysis the result execute from the located function.
str.push(): push the string result data into the information window.

AMap.Weather():
Weather query service returns city weather forecast information, including real-time weather information, based on city name or area code.
getLive('city/district'): according to location, search the real-time weather.

```
//Analysis of the location results
function onComplete(data) {

    document.getElementById('status').innerHTML='Successfully located'
    var str = [];
    str.push('Located result: ' + data.position);
    str.push('Located types: ' + data.location_type);
    if(data.accuracy){
        str.push('Accuracy: ' + data.accuracy + ' meters');
    }//If IP is accurate, no need for accuracy
    str.push('Shift or not: ' + (data.isConverted ? 'Yes' : 'No'));
    document.getElementById('result').innerHTML = str.join('<br>');

    AMap.plugin('AMap.Weather', function() {
    var weather = new AMap.Weather();
    //Inquire of real-time weather, the city/district name is optional
    weather.getLive(value6, function(err, data) {
        if (!err) {
            var str = [];
            str.push('<h4 >Real-time weather: ' + '</h4><hr>');
            str.push('<p>City/District: ' + data.city + '</p>');
            str.push('<p>Weather: ' + data.weather + '</p>');
            str.push('<p>Temperature: ' + data.temperature + '℃</p>');
            str.push('<p>Wind direction: ' + data.windDirection + '</p>');
            str.push('<p>Wind Strength: ' + data.windPower + ' Level</p>');
            str.push('<p>Humidity: ' + data.humidity + '</p>');
            str.push('<p>Released time: ' + data.reportTime + '</p>');
            var marker = new AMap.Marker({map: map, position: map.getCenter()});
            var infoWin = new AMap.InfoWindow({
                content: '<div class="info" style="position:inherit;margin-bottom:0;">'+str.join('')+'</d
                isCustom:true,
                offset: new AMap.Pixel(0, -37)
            });
            infoWin.open(map, marker.getPosition());
            marker.on('mouseover', function() {
                infoWin.open(map, marker.getPosition());
            });
        }
    });
});
}
```

onError(): Output error if met.

```
//Analysis of the location failure
function onError(data) {
    document.getElementById('status').innerHTML='Fail to locate'
    document.getElementById('result').innerHTML = 'Reasons of failure:'+data.message+'</br>Callback information from browser: '+data.originMessage;
}
```

AMap.StationSearch():
Bus stop query service, queries bus stop information based on input keywords and IDs. Users can retrieve and display query results through custom callback functions.

station.search():
Query bus stop details based on given bus stop names; multiple keywords are separated by "|". Through getting the value of 'stationKeyWord' by input id, the relevant located bus stops will be searched and mark.

stationSearch_CallBack():
Function used to demonstrate the markers of bus stops and manage the callback result.

JSON.stringify():
Format of data send to web server must be string, the function can transform any data type in JavaScript into string and send to server.

```
var markers = [];
stationSearch();

//bus stop searching
function stationSearch() {

    // Remove all the Marker
    map.remove(markers);

    markers = [];
    var stationKeyWord  = document.getElementById('stationKeyWord').value
    if(!stationKeyWord) return

    //Instantiate the bus stop query class
    var station = new AMap.StationSearch({
        pageIndex: 1, //page number
        pageSize: 60, //display the number of results per page
        city: ''   //ensure the search is within the city
    });

    station.search(stationKeyWord, function(status, result) {

        if (status === 'complete' && result.info === 'OK') {
            stationSearch_CallBack(result);
        } else {
            document.getElementById('tap').innerHTML = JSON.stringify(result);
        }
    });

}
```

stationSearch_CallBack():

Monitor all the event and function retrieve from stationSearch(), and add icons on position of bus stops.

AMap.Marker():

Construct a point marker object and set the properties of the point marker object through MarkerOptions.

offset: new AMap.Pixel():

Add offset by setting offset. When the offset is (0, 0), the custom Marker defaults to aligning the upper left corner with the latitude and longitude coordinates.

maker.on():

Register events, register events for Map or overlay objects.

```
//analysis of the search results of bus stop
function stationSearch_CallBack(searchResult) {

    var stationArr = searchResult.stationInfo;
    var searchNum = stationArr.length;

    if (searchNum > 0) {
        document.getElementById('tap').innerHTML = 'Inquire results: ' + searchNum + 'related bus stops';
        for (var i = 0; i < searchNum; i++) {

            var marker = new AMap.Marker({
                icon:new AMap.Icon({
                   image:'//a.amap.com/jsapi_demos/static/resource/img/pin.png',
                   size:new AMap.Size(32,32),
                   imageSize:new AMap.Size(32,32)
                }),

                offset:new AMap.Pixel(-16,-32),
                position: stationArr[i].location,
                map: map,
                title: stationArr[i].name
            });

            marker.info = new AMap.InfoWindow({
                content: stationArr[i].name,
                offset: new AMap.Pixel(0, -30)
            });

            marker.on('mouseover', function(e) {
                e.target.info.open(map, e.target.getPosition())
            })

            markers.push(marker);
        }

        map.setFitView();
    }
}

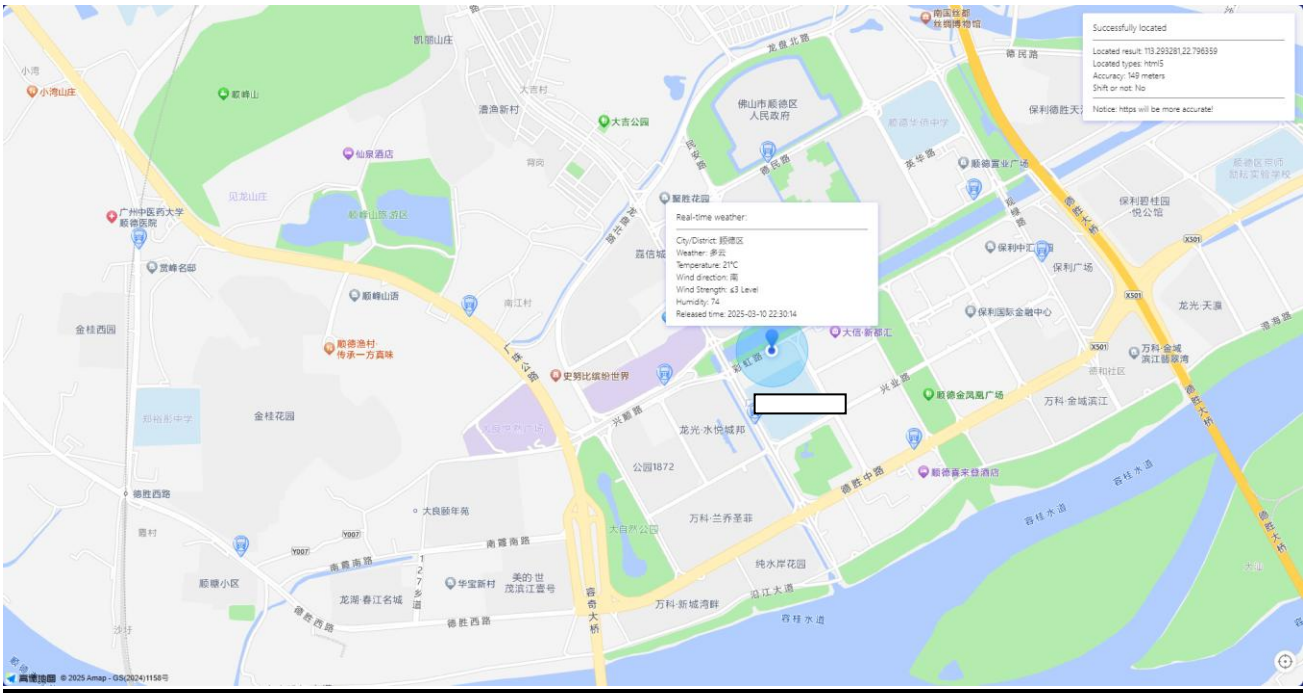
document.getElementById('tip').onclick = stationSearch;
```

IP (weather and bus stop):



IP (weather and bus stop):

Locate users IP address through the browsers they are using, and show surrounding bus stops.



Wordcount: 1364

References

Sam Meech-Ward. (2022, June 22). MySQL Node.js Express. Www.youtube.com. https://www.youtube.com/watch?v=Hej48pi_lOc

Reference Manual - Map JS API 2.0 | Amap API. (2024). Amap.com. https://lbs.amap.com/api/javascript-api-v2/documentation#walkingclear

Webslesson. (2022, April 20). *Node JS Tutorial | Fetch & Display Data From MySQL Database*. YouTube. https://www.youtube.com/watch?v=SnncAvMYxgY

MYSQL. (2024). *MySQL :: MySQL 8.4 Reference Manual :: 15.2.13 SELECT Statement*. Mysql.com. https://dev.mysql.com/doc/refman/8.4/en/select.html